MODULE 12 · DOMAIN 2 - RELIABILITY AND BUSINESS CONTINUITY

Auto Scaling in Depth

Policies, health checks, warm pools, lifecycle hooks and instance refresh. The start of domain two.

DOMAIN 2 of 5 EXAM WEIGHT 22% MODULE 12 of 35

admin@sthithapragna.com

An Auto Scaling group is a **control loop**. Policies move desired capacity; everything else is about **timing** - how fast, how long to wait, when to judge health.

Launch templates, and the end of launch configurations

- **Launch templates are versioned.** You can roll forward and roll back, and an ASG points at a specific version or at Latest.
- **Launch configurations are legacy.** No new EC2 instance types since 1 January 2023.
- **Accounts created on or after 1 October 2024 cannot create one at all**, by console, CLI, API or CloudFormation.
- Only launch templates support **mixed instances, Spot alongside On-Demand, T unlimited mode, placement groups and IMDSv2 defaults**.

> ON THE EXAM
>
> If an option proposes a launch configuration, it is either wrong or describing a legacy estate. Templates are the only supported choice for new work.

The group itself

- **Minimum, maximum and desired capacity.** Policies change desired; the group never goes outside min and max.
- Spread across **multiple Availability Zones**, and the group rebalances automatically if one AZ becomes uneven.
- **AZ rebalancing launches before terminating**, so it can briefly exceed desired capacity. That is normal.
- **Group metrics are off by default.** GroupInServiceInstances and friends must be enabled, as module one covered.

> ON THE EXAM
>
> A scaling policy that appears to do nothing is often a group already at its maximum. Check the ceiling before the policy.

Health checks, and the grace period trap

- **EC2 health checks** are the default. They only confirm the instance is running and passing status checks.
- **ELB health checks** ask the load balancer whether the target is actually serving. Enable these for anything behind a load balancer.
- **Custom health checks** let your own code mark an instance unhealthy through SetInstanceHealth.
- **The health check grace period** is how long after launch before checks count. Too short and instances are killed mid-boot, forever.

> ON THE EXAM
>
> "Instances launch, terminate, and launch again in a loop" is a grace period shorter than the application's start-up time. Nothing else produces that pattern.

Five ways to scale

POLICY TYPE	USE IT WHEN
Target tracking	You know the metric value you want to hold. The default choice
Step scaling	You want different responses to different severities of breach
Simple scaling	Legacy. One adjustment, then a cooldown. Rarely the answer
Scheduled	Load changes by the clock: business hours, batch windows, a known event
Predictive	Recurring daily or weekly patterns, and scaling out reactively is too slow

ON THE EXAM

Target tracking unless the scenario gives you a reason not to. Scheduled and predictive can run alongside a dynamic policy, and normally should.

Target tracking or step scaling?

TARGET TRACKING

Hold this number

- Pick a metric and a target, such as 50% CPU
- AWS creates and manages the CloudWatch alarms
- Scales out and in to keep the metric near target
- Simplest to reason about and to review

STEP SCALING

Respond in proportion

- Steps defined by how far past the threshold you are
- Add 2 instances at 60%, add 6 at 85%
- You own the alarms and the steps
- For sharp, uneven load spikes

THE TIEBREAKER

Target tracking is a thermostat. Step scaling is a set of rules for how hard to react. If the scenario does not describe varying severity, it wants target tracking.

Target tracking, in detail

- Predefined metrics: **average CPU**, **average network in or out**, and **ALB request count per target**.
- Custom metrics and metric math work too, so you can track queue depth per instance or a business metric.
- You may have **several target tracking policies** at once. The group scales out to satisfy **whichever demands most**.
- **Disable scale-in** when something else, such as a scheduled policy, should own scaling down.

> ON THE EXAM
>
> ALBRequestCountPerTarget is the underrated one. It tracks demand directly rather than the CPU symptom of demand.

Cooldown or warm-up? A real distinction

- **Cooldown** is a pause after a scaling activity before another may start. It belongs to **simple scaling**, and defaults to 300 seconds.
- **Instance warm-up** is how long a new instance is excluded from the group's metrics while it starts up.
- **Target tracking and step scaling use warm-up, not cooldown.** They can keep scaling while earlier instances are still warming.
- Warm-up set too low means new instances drag the average down before they are ready, which causes **over-scaling then a sharp scale-in**.

> ON THE EXAM
>
> Cooldown blocks further scaling. Warm-up only hides a new instance's metrics. Confusing them produces either a sluggish group or a thrashing one.

Scheduled and predictive scaling

- **Scheduled actions** set minimum, maximum or desired capacity at a time you choose. Cron expressions support a **local time zone**, so daylight saving is handled.
- **Predictive scaling** forecasts up to 48 hours ahead from history and scales out **before** demand arrives.
- **Predictive scaling only scales out.** Pair it with target tracking, which handles scale-in and unexpected spikes.
- Start in **ForecastOnly** mode to review predictions, then switch to **ForecastAndScale**.

> ON THE EXAM
>
> "Traffic ramps faster than instances can boot" is predictive scaling. Reactive scaling cannot beat a spike that outruns your start-up time.

The instance lifecycle

01
Pending
Launching. A launch lifecycle hook holds it in Pending:Wait.

›

02
InService
Serving traffic. Health checks apply after the grace period.

›

03
Standby
Optional. Out of service for maintenance, not replaced.

›

04
Terminating
A termination hook holds it in Terminating:Wait.

›

05
Terminated
Or returned to the warm pool, if reuse is enabled.

ON THE EXAM

Hooks act at the two Wait states. Standby is the manual detour. Warm pool reuse replaces the final step.

Lifecycle hooks

- Pause an instance in **Pending:Wait** before it joins, or **Terminating:Wait** before it goes, while your automation does something.
- Launch uses: install software, register with a configuration system, warm a cache, pull data.
- Termination uses: drain connections, upload logs, deregister from a service, take a final snapshot.
- Default timeout is **one hour**. Call **CompleteLifecycleAction** when done, or send heartbeats to extend it.

> ON THE EXAM
>
> "Upload the logs before the instance disappears" is a termination lifecycle hook. Nothing else gives you a window after the decision to terminate.

Warm pools

- A pool of **pre-initialised instances** kept outside the group, usually **Stopped**, sometimes Running or Hibernated.
- On scale-out the group takes an instance from the pool instead of launching cold, so start-up drops from minutes to seconds.
- You pay for **storage but not compute** while instances sit stopped, which is why the pattern is affordable.
- Instances can be **returned to the pool on scale-in** rather than terminated, so the initialisation work is not repeated.

> ON THE EXAM
>
> Warm pools are for applications with long start-up: large AMIs, heavy runtimes, big caches to populate. They do nothing for a fast-booting service.

Warm pool or a better AMI?

WARM POOL WHEN

Initialisation cannot be baked

- Cache warming, data loading, licence checkout
- Work that depends on current data
- Start-up is genuinely minutes
- Adds a pool to manage and pay storage for

GOLDEN AMI WHEN

Initialisation is installation

- Installing packages, runtimes, agents, application code
- EC2 Image Builder produces it on a schedule
- No ongoing cost beyond snapshot storage
- Fix start-up once, for every future instance

THE TIEBREAKER

Ask what the boot time is spent on. Installing software belongs in the image. Loading today's data does not.

Instance refresh

- Rolling replacement of every instance in the group, to pick up a new launch template version or AMI.
- **MinHealthyPercentage** controls how much capacity may be lost. Setting it to **100** gives **launch-before-terminate**.
- **Checkpoints** pause at chosen percentages with a bake time, so you can watch before continuing.
- **Skip matching** leaves instances that already have the desired configuration alone. **Alarm-based auto-rollback** reverses a bad rollout.

> ON THE EXAM
>
> Scaling policies and health checks stay active during an instance refresh, unlike the old rolling update. The group can still scale mid-deployment.

New: instance refresh as a CloudFormation update policy

- **Announced 29 July 2026.** The **AutoScalingInstanceRefresh** update policy on AWS::AutoScaling::AutoScalingGroup.
- CloudFormation triggers a refresh automatically when you change the launch template or mixed instances policy.
- It is **mutually exclusive with AutoScalingRollingUpdate**, the older policy, which suspends scaling and ignores your maintenance policy.
- Brings checkpoints, bake time, launch-before-terminate and alarm rollback into the stack update itself. No extra cost.

> ON THE EXAM
>
> Before this, teams either accepted rigid rolling updates or ran refreshes outside the stack and created drift. This closes that gap.

Which instance gets terminated

- **Default policy**: balance across Availability Zones first, then oldest launch template or configuration, then closest to the next billing hour.
- Alternatives include **OldestInstance**, **NewestInstance**, **AllocationStrategy** for Spot, or a custom ordered list.
- **Instance scale-in protection** exempts specific instances, for example one running a long job.
- **Standby** removes an instance from service for maintenance without the group replacing it.

> ON THE EXAM
>
> "A long-running job was killed by a scale-in" has two answers: scale-in protection for the instance, or a termination lifecycle hook to let it finish.

Suspending processes

- Individual group processes can be **suspended and resumed**: Launch, Terminate, HealthCheck, ReplaceUnhealthy, AZRebalance, AlarmNotification, ScheduledActions, AddToLoadBalancer, InstanceRefresh.
- Suspending **Terminate** and **ReplaceUnhealthy** during an investigation stops the group destroying the evidence.
- **Suspending Launch also blocks scale-out**, and suspending AddToLoadBalancer means new instances never receive traffic.
- **AWS itself suspends processes** when a group repeatedly fails to launch. A group that has stopped scaling may be suspended, not misconfigured.

> ON THE EXAM
>
> "The group has stopped scaling and nobody changed anything" is worth checking here before anywhere else.

Mixed instances and Spot

- A **mixed instances policy** lets one group use several instance types, and On-Demand and Spot together.
- Set an **On-Demand base capacity** for the part that must never be interrupted, then a percentage split above that.
- **price-capacity-optimized** is the recommended Spot allocation strategy: cheap, from the deepest capacity pools.
- **Attribute-based instance type selection** expresses requirements as vCPU and memory ranges, so new instance types are used automatically.

> ON THE EXAM
>
> Capacity Rebalancing acts on a Spot rebalance recommendation, replacing an instance before it is interrupted rather than after.

Application Auto Scaling is a different service

- **EC2 Auto Scaling** scales EC2 instances in an Auto Scaling group. That is all it does.
- **Application Auto Scaling** scales everything else: **ECS services, DynamoDB tables and indexes, Aurora replicas, Lambda provisioned concurrency, SageMaker endpoints, Spot Fleet**.
- Same policy concepts: target tracking, step scaling and scheduled scaling.
- **Aurora Serverless v2 and DynamoDB on-demand scale themselves**, and need no policy at all.

> ON THE EXAM
>
> "Scale the ECS service when the queue grows" is Application Auto Scaling, not EC2 Auto Scaling. Getting the service name right matters here.

"The group did not scale" checklist

- **Is desired already at maximum?** The policy fires, requests more, and the ceiling refuses. Most common cause.
- **Are processes suspended?** Either by you, or automatically after repeated launch failures.
- **Is the alarm actually in ALARM?** Module two applies: M out of N, missing data, and a period shorter than the publishing interval.
- **Is capacity available?** An instance type with no capacity in that AZ, or an exhausted subnet with no free IP addresses, fails silently at launch.

> ON THE EXAM
>
> Scaling activity history on the group records every attempt and its failure reason. Read that before theorising.

A Java application takes about four minutes to start and pass its health check. An Auto Scaling group behind an ALB uses ELB health checks with a health check grace period of 120 seconds.

Instances launch, are marked unhealthy, are terminated and replaced. The group never reaches its desired capacity, and this repeats indefinitely.

What is the cause and the fix?

A The ALB target group's health check threshold is too strict. Increase the unhealthy threshold

B The health check grace period is shorter than start-up time. Increase it beyond four minutes

C The instance warm-up is too low. Increase it to 300 seconds

D Switch the group to EC2 health checks instead of ELB health checks

◆ PAUSE AND DECIDE

What is the cause and the fix?

A Raise the unhealthy threshold

Would delay the verdict a little, and 120 seconds against a four-minute start-up is not a threshold that tuning can rescue.

B Increase the grace period

The grace period is precisely how long after launch before health checks count. It is set to half the application's start-up time.

C Increase instance warm-up

Warm-up excludes a new instance from scaling metrics. It has no effect on whether health checks terminate it.

D Use EC2 health checks

Hides the problem by no longer checking whether the application serves traffic. The loop stops and real failures stop being detected too.

THE SIGNAL

Launch, terminate, launch again, forever, is the signature of a grace period shorter than start-up. Two numbers in the setup are all you need.

An online retailer sees traffic climb steeply from 08:00 every weekday, reaching four times its overnight level within twenty minutes. Instances take six minutes to become useful. Target tracking on CPU is configured, and customers still see errors between 08:00 and 08:30 every day.

The pattern has been consistent for over a year.

What should the team add?

A. Lower the target tracking CPU target from 60 percent to 40 percent

B. Predictive scaling in ForecastAndScale mode, keeping target tracking for unexpected spikes

C. A step scaling policy with more aggressive steps

D. Increase the group's maximum capacity

◆ PAUSE AND DECIDE

What should the team add?

A. Lower the CPU target

Reacts sooner and still only after load arrives, then runs excess capacity for the other twenty-two hours of the day.

B. Predictive scaling plus target tracking

A year of consistent history is exactly what the forecast needs, and capacity arrives before 08:00 rather than six minutes after demand.

C. More aggressive step scaling

Still reactive. However aggressive the steps, instances still need six minutes, and the ramp is twenty minutes long.

D. Increase maximum capacity

Raises the ceiling. Nothing in the scenario says the ceiling was reached; the problem is arrival time, not the limit.

THE SIGNAL

Every reactive answer loses the same race: six minutes to boot against a twenty-minute ramp. Only scaling before 08:00 wins it.

A video processing fleet runs jobs that take up to 40 minutes. During scale-in, instances are terminated mid-job and the work is lost. The team also needs each instance to upload its logs to S3 before it disappears.

Jobs cannot be made resumable in the near term.

Which combination should the team implement?

A. Instance scale-in protection while a job is running, plus a termination lifecycle hook to upload logs

B. Set the termination policy to NewestInstance and increase the cooldown

C. Disable scale-in on the target tracking policy and increase the health check grace period

D. Move the instances to a warm pool when they finish a job

◆ PAUSE AND DECIDE

Which combination should the team implement?

A Scale-in protection plus a termination hook

Protection stops a busy instance being chosen at all; the hook gives a window after the decision to upload logs. Two requirements, two mechanisms.

B NewestInstance plus cooldown

Changes which instance dies, not whether it is busy. The newest instance can be mid-job just as easily.

C Disable scale-in

Solves it by never scaling in, so the fleet only grows. It also does nothing about uploading logs.

D Warm pool on job completion

Warm pools accelerate scale-out. They are not a mechanism for protecting running work or for draining it.

THE SIGNAL

Two requirements again: do not kill busy instances, and do something before an instance dies. Protection answers the first, a lifecycle hook the second.

A team must deploy a new AMI to a 60-instance Auto Scaling group during business hours. Capacity must never drop below current levels. The rollout must pause for review after the first quarter, and reverse itself automatically if error rates rise.

The group is defined in CloudFormation and the team wants the stack to stay consistent with reality.

What should the team use?

A The AutoScalingRollingUpdate CloudFormation update policy with MaxBatchSize set to 15

B The AutoScalingInstanceRefresh update policy with MinHealthyPercentage 100, checkpoints and an alarm specification

C Run an instance refresh manually from the console after updating the stack

D Create a second Auto Scaling group with the new AMI and shift the ALB weighting

◆ PAUSE AND DECIDE

What should the team use?

A AutoScalingRollingUpdate

No checkpoints, no alarm rollback, and it terminates before launching, so capacity dips. It also suspends scaling policies during the update.

B AutoScalingInstanceRefresh policy

MinHealthyPercentage 100 gives launch-before-terminate, checkpoints give the review pause, and the alarm specification gives automatic rollback, all inside the stack.

C Manual refresh after the stack update

Gets the deployment right and breaks the requirement that the stack stays consistent. The refresh happens outside CloudFormation's knowledge.

D Second group with ALB weighting

A valid blue/green pattern, and it doubles the fleet, duplicates the stack and is far more machinery than the requirements ask for.

THE SIGNAL

Four requirements, and the July 2026 update policy is the only option that meets all four without leaving CloudFormation.

An Auto Scaling group stopped scaling out three days ago. CPU regularly exceeds the target tracking threshold and the alarm is firing correctly, but desired capacity never changes. The group is at 12 instances with a maximum of 40.

Nobody has modified the group. Scaling activity history shows repeated failed launches ending in an error about available IP addresses.

What is the most likely explanation?

A The subnets are exhausted, launches failed repeatedly, and Auto Scaling has suspended the Launch process

B The target tracking policy needs its cooldown reduced

C The group has reached its maximum capacity

D The health check grace period is too short

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Subnet exhaustion, then automatic suspension

The activity history names the cause, and AWS suspends processes after repeated launch failures. That is why it stopped rather than merely failing.

B Reduce the cooldown

Cooldown belongs to simple scaling and would delay scaling, not prevent it entirely for three days.

C At maximum capacity

The first thing to check, and the scenario has already ruled it out: 12 instances against a maximum of 40.

D Grace period too short

That produces a launch-terminate loop with instances constantly appearing. Here nothing launches at all.

THE SIGNAL

Two facts are handed to you: 12 of 40, and an IP address error in the activity history. Together they eliminate everything except suspension.

The five things to carry forward

- **Launch, terminate, repeat forever** is a health check grace period shorter than start-up time.
- **Cooldown blocks scaling; warm-up hides a new instance's metrics.** Target tracking and step scaling use warm-up.
- **Reactive scaling cannot beat a ramp faster than boot time.** That is what predictive scaling is for, and it only scales out.
- **Scale-in protection stops the choice; a lifecycle hook creates a window after it.**
- **AutoScalingInstanceRefresh (July 2026)** brings checkpoints, launch-before-terminate and alarm rollback into CloudFormation.

> ON THE EXAM
>
> Next: load balancing, and the error codes that tell you exactly which part of the path is broken.